

Fully Sharded Data Parallel (FSDP)

Personal lecture notes — construction, forward/backward mechanics, communication cost, and the data-parallel scaling ceiling

Sources: arXiv 2304.11277 (PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel, Zhao et al., Meta AI); PyTorch FSDP1/FSDP2 tutorials; An Empirical Model of Large-Batch Training (McCandlish et al., OpenAI).

0. Why FSDP exists

Plain data parallelism (DDP) replicates the full model — parameters, gradients, and optimizer states — on every GPU. This is fine until the model gets big enough that one GPU cannot hold a full copy plus activations.

Worked example: when does DDP break?

A 5B-parameter model in bf16:

- Parameters: ~10 GB (bf16, 2 bytes/param)
- Gradients: ~10 GB (bf16, 2 bytes/param)
- Adam optimizer states: ~40 GB (fp32 master weights + momentum + variance, ~4x params in fp32 terms)

Total: ~60 GB before activations — already at or beyond an 80 GB H100's budget, and this cost is paid identically on every single GPU under DDP, regardless of how many GPUs you have.

Note: This is a memory problem, not a compute problem. Adding more GPUs under DDP does not help — every GPU still needs the full 60 GB.

FSDP's fix

Instead of every GPU holding 100% of params/gradients/optimizer states, FSDP shards all three across the N GPUs, so each GPU holds only 1/N of each. For 8 GPUs, that ~60 GB drops to roughly ~7.5 GB per GPU (before activations).

1. Construction: sharding at rest

Before any training step runs, FSDP does one-time setup work:

1. Group the model into FSDP units — commonly one unit per transformer block.
2. Within a unit, flatten every parameter tensor (Q, K, V projections, output projection, FFN matrices, ...) end-to-end into a single 1D tensor — the FlatParameter.
3. Chunk that FlatParameter into N equal-length pieces (N = number of GPUs) and give each GPU exactly one piece, permanently.

The GPT-2 mental model

Take a 12-block GPT-2, 8 GPUs. Each block's 6 weight tensors get concatenated into one long ribbon, then sliced into 8 equal segments — like cutting a rope with scissors, ignoring where the original tensor boundaries fall.

```
Block 1: [Q|K|V|Out|FFN1|FFN2] -> one long ribbon
|-s1-|-s2-|-s3-|-s4-|-s5-|-s6-|-s7-|-s8-|
GPU0 GPU1 GPU2 GPU3 GPU4 GPU5 GPU6 GPU7
```

```
Block 2: own ribbon, sliced the same way
```

```

...
Block 12: own ribbon, sliced the same way

End state: every GPU holds 12 thin slices
(one per block). No GPU holds a full block.

```

Why flatten-then-slice instead of slicing each tensor individually?

A single long ribbon can be cut into N perfectly equal pieces trivially. Six oddly-shaped tensors sliced into N even pieces each would be messy to reconstruct and produces many small, inefficient NCCL collectives instead of one large efficient one.

How does FSDP avoid losing track of what's what?

It doesn't track shards at the level of "this row belongs to that tensor." Each GPU's shard is dumb, meaningless storage — just numbers at flat positions. Separately, a tiny piece of shape metadata (tensor name, shape, start/end offset in the ribbon) is kept identically on every GPU. Reconstruction just re-slices the gathered ribbon using that metadata.

$$\text{shard_}i = \text{FlatParameter}[i \times (L/N) : (i+1) \times (L/N)], \quad i = 0 \dots N-1, \quad L = \text{ribbon length}$$

2 & 3. Forward pass: all-gather, compute, discard

When block k 's forward computation is about to run, every GPU only holds $1/N$ of it. FSDP performs an all-gather: all N GPUs simultaneously exchange their shard of block k 's ribbon, and every GPU ends up holding the complete, identical, unsharded block k — reconstructed into its original named tensors via the shape metadata.

- All GPUs become redundant on weights (identical full copy) but stay different on data (each GPU still has its own batch of tokens).
- Compute runs exactly as it would on a single, non-sharded GPU — sharding is invisible to the math.
- The instant block k 's output is produced, the borrowed $N-1/N$ of the block is discarded on every GPU, falling back to just the permanent $1/N$ shard.
- Only one block's full weights exist in memory at any instant — never all blocks simultaneously.

```

Block 1:  gather -> compute -> DISCARD
Block 2:  gather -> compute -> DISCARD
...
Block 12: gather -> compute -> DISCARD -> loss

```

Note: Between blocks, the other 11 blocks' shards are quietly resident on GPU (still sharded, $1/N$ each) — not offloaded to CPU/SSD unless CPUOffload is explicitly enabled.

4. Backward pass: all-gather again

Backward propagation runs block 12 -> block 1. Because every block's full weights were discarded during forward, FSDP must all-gather each block a second time before it can compute that block's gradients.

```

Forward:  1->2->3->...->12 (gather once)
Backward: 12->11->10->...->1 (gather AGAIN)

Net: every block is gathered twice per step.

```

Note: This double-gather is a major reason FSDP's total communication volume exceeds DDP's — quantified in Section 8.

5. Backward pass: gradients and reduce-scatter

With block k 's full weights back in memory, each GPU computes gradients as normal — but since each GPU processed different tokens, each GPU produces a different full gradient for the same block. These must be summed across GPUs, and each GPU only needs to retain its own $1/N$ slice of the result going forward.

FSDP uses reduce-scatter instead of DDP's all-reduce: sum all GPUs' gradients together, then immediately split the summed result and hand each GPU only its $1/N$ slice — never materializing the full summed gradient on any single GPU.

$$\text{grad_shard}_i = (\sum \text{grad_full}_j) [i \times (L/N) : (i+1) \times (L/N)]$$

Who coordinates this? (no leader)

Reduce-scatter, like all-reduce, is a collective operation executed symmetrically by all GPUs as equal peers — there is no leader/boss GPU. Under NCCL, a ring or tree algorithm passes partial sums between neighboring GPUs over several rounds until each GPU ends up holding its correct final slice. No GPU ever materializes the full summed result.

6. Optimizer step — local and communication-free

After backward, each GPU holds exactly matching $1/N$ slices of both parameters and gradients for a given block — the same shard of parameters it has held since construction, and now the matching shard of gradients from reduce-scatter.

The optimizer (e.g. Adam) runs its normal update math directly on this local $1/N$ slice. No communication is needed for this step at all — every GPU updates its own shard independently, in parallel.

Note: This is also why optimizer state (momentum, variance in Adam) only costs $1/N$ per GPU — the biggest single memory win in FSDP.

7. Overlapping communication with compute

GPUs sitting idle waiting for a network transfer wastes the interconnect and the compute unit simultaneously. FSDP overlaps them:

- Forward: while block k computes, block $k+1$'s all-gather is prefetched in the background.
- Backward: same prefetching pattern, plus reduce-scatter (pure communication, no compute) is run on a separate CUDA stream so it overlaps with the next block's gradient computation rather than blocking.

This doesn't reduce the total data moved — it hides the latency behind useful compute, so a "busy network" doesn't necessarily mean idle GPUs.

8. The real cost: FSDP vs DDP communication volume

DDP

One collective per step: a single all-reduce on gradients. A ring all-reduce moves roughly $2\times$ the parameter count in traffic.

$$\text{DDP communication} \approx 2P \quad (P = \text{parameter count})$$

FSDP

Three collectives per step, each moving roughly $1 \times$ the parameter count:

- Forward-pass all-gather $\approx 1P$
- Backward-pass all-gather $\approx 1P$
- Backward-pass reduce-scatter $\approx 1P$

$$FSDP \text{ communication} \approx 3P$$

The trade

$$3P / 2P = 1.5 \times \text{ — } FSDP \text{ moves } \sim 50\% \text{ more data than DDP}$$

This is a deliberate trade: $\sim 50\%$ more network traffic in exchange for needing only $1/N$ the memory per GPU for parameters, gradients, and optimizer states.

- Below the memory cliff (model fits under DDP): DDP is strictly cheaper on communication — prefer DDP.
- Above the memory cliff (DDP would OOM): FSDP's extra communication is the price of being able to train at all. The overlap tricks in Section 7 keep this from translating one-to-one into extra wall-clock time.

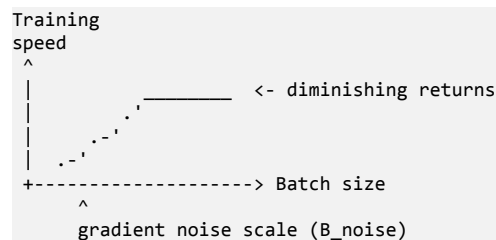
9. The other ceiling: gradient noise scale

FSDP solves the memory problem. It does not solve a separate, independent problem: the compute-scaling ceiling from data parallelism itself (from McCandlish et al., "An Empirical Model of Large-Batch Training").

Core idea

Every individual example's gradient is a noisy estimate of the true direction weights should move in. Averaging across a batch reduces that noise — but with diminishing returns, not linearly.

- Below the gradient noise scale: more batch (more GPUs, more data per step) gives close to linear speedup, because each additional example still meaningfully reduces noise in the gradient estimate.
- Above the gradient noise scale: additional examples are not duplicated data — they are genuinely new tokens — but they add only a shrinking amount of new directional information, because the batch already captured most of the useful signal about which way to step.



Why FSDP doesn't change this

FSDP is still fundamentally data-parallel — different tokens per GPU, gradients combined at the end. It inherits this ceiling exactly like DDP does. FSDP lets you fit a bigger model in memory; it does not let you usefully add GPUs past the noise scale for a fixed model and fixed data.

Practical levers past the noise ceiling (5-hour, $50 \times H100$ scenario)

- Use additional GPUs for tensor/pipeline parallelism to speed up each individual step, rather than widening the data-parallel batch further.
- LR warmup and batch-size scaling rules (from the same noise-scale paper) squeeze more out of a given batch size without eliminating the ceiling.
- Mixed precision, fused kernels, and communication/compute overlap (Section 7) often buy more real wall-clock speedup than adding raw GPU count once past the ceiling.

Estimating an optimal GPU / batch size

The noise scale can be measured empirically early in training by tracking how true gradient variance changes with batch size. The batch size that lands near this measured noise scale is roughly the sweet spot — below it, more batch/GPUs pays off close to linearly; above it, it doesn't. In practice the noise scale grows as training progresses, so large-scale runs sometimes increase batch size over the course of training to track it.

10. One-page summary

Step	What happens
1. Construction	Flatten each unit's tensors into one ribbon, slice into N equal shards, one per GPU, permanent at rest.
2–3. Forward	All-gather block k -> full weights on every GPU -> compute -> discard, repeat block by block.
4. Backward gather	All-gather block k again (12'11...→1) since it was discarded after forward.
5. Backward grad	Reduce-scatter: sum gradients across GPUs, each GPU keeps only its 1/N slice.
6. Optimizer	Local update on matching 1/N parameter + gradient shards — zero communication.
7. Overlap	Prefetch next block's gather during current compute; reduce-scatter on separate CUDA stream.
8. Cost	FSDP $\approx$ 3P communication vs DDP $\approx$ 2P — $\sim 1.5\times$ more traffic, traded for 1/N memory per GPU.
9. Noise ceiling	Separate, unsolved-by-FSDP limit: past the gradient noise scale, more GPUs/batch stops giving proportional speedup.